Flask-SQLAlchemy : Code-Along (CodeGrade)

 (https://github.com/learn-co-curriculum/python-p4-v2-flask-sqlalchemy-1-intro)  (https://github.com/learn-co-curriculum/python-p4-v2-flask-sqlalchemy-1-intro/issues/new)

Learning Goals

- Define a Python model class.
- Use **Flask-SQLAlchemy** to configure a Flask app to connect to a database.
- Use **Flask-SQLALchemy** to define a database model.
- Use **Flask-Migrate** to initialize a database schema.

Key Vocab

- **Object-Relational Mapping (ORM):** A technique of accessing a relational database using an object-oriented programming language.
- **Schema:** The blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
- **SQLAlchemy:** An open-source SQL library and object-relational mapper (ORM) for the Python programming language.
- **Flask-SQLAlchemy:** A Flask extension that makes it easier to use SQLAlchemy.
- **Schema Migration:** - A schema migration is performed on a database whenever it is necessary to update or revert that database's schema to some newer or older version.
- **Database Migration Tool:** - Software that manages version-controlled, incremental and reversible changes to relational database schemas.
- **Alembic:** A lightweight database migration tool for usage with SQLAlchemy.
- **Flask-Migrate:** A Flask extension for managing database migrations.

Introduction

Flask is a micro web framework written in Python. We've seen how to use Flask to implement a web server with basic routing and views that handle user requests.

Flask has no database abstraction layer, form validation, authorization, or other common web components that integrate pre-existing third-party libraries. However, Flask supports extensions that can add these useful features to a web server.

Recall that Object Relational Mapping (ORM) is a way for our Python programs to manage database data by "mapping" database tables to classes and instances of classes to rows in those tables. **SQLAlchemy** is an open-source SQL library and object-relational mapper (ORM) for the Python programming language. **Flask-SQLAlchemy** is a Flask extension that makes it easier to work with SQLAlchemy.

A **schema migration** is performed whenever it is necessary to update or revert the database schema to a newer or older version. **Alembic:** is a lightweight database migration tool for SQLAlchemy. **Flask-Migrate** is a Flask extension that handles database migrations using Alembic.

In this lesson, we will use Flask-SQLAlchemy to configure a database connection and to define a model for storing data about pets. We will use Flask-Migrate to perform an initial migration to create a database containing the pets table.

Setup

This lesson is a code-along, so fork and clone the repo.

`Pipfile` has some new dependencies that we'll use in this lesson: `flask-sqlalchemy` and `flask-migrate`. Run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
$ pipenv install
$ pipenv shell
```

Let's use the `tree` command to view the directory structure. MacOS users can install this with the command `brew install tree`. WSL and Linux users can run `sudo apt-get install tree` to download it.

```
$ tree
```

The `server` folder initially contains two files, `app.py` and `models.py`

```
├── CONTRIBUTING.md
├── LICENSE.md
├── Pipfile
├── Pipfile.lock
├── README.md
└── server
    ├── app.py
    └── models.py
```

- `app.py` contains the code to configure and connect a web server to a database.
- `models.py` contains a **model** named `Pet`.

Defining A Model

A **model class**, also referred to simply as a **model**, is a Python class that (1) defines a new Python data type, and (2) defines database metadata to describe an SQL table. We can treat a model class like any other Python class, meaning we can create instances, invoke methods, etc. Additionally, Flask-SQLAlchemy performs object-relational mapping between the model class and an associated database table, while Flask-Migrate uses changes to the model class to evolve a database schema.

Defining a model with Flask-SQLAlchemy requires a Python class with four key traits:

- Inheritance from the `db.Model` class.
- A `__tablename__` class attribute.
- One or more class attributes assigned to the type `db.Column`.
- One class attribute specified to be the table's primary key.

Let's take a look at a model defined by the Python class `Pet` in `server/models.py`:

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import MetaData

# contains definitions of tables and associated schema constructs
# read more about Metadata using the link at the bottom of the page
metadata = MetaData()

# create the Flask SQLAlchemy extension
db = SQLAlchemy(metadata=metadata)

# define a model class by inheriting from db.Model.
class Pet(db.Model):
    __tablename__ = 'pets'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String)
    species = db.Column(db.String)
```

- The `Pet` class is declared as a subclass of `db.Model` .
- The database table is named `pets` .
- The database table has 3 columns:
  - the `id` column is the primary key
  - the `name` column stores a string
  - the `species` column stores a string

Configure the Flask-SQLAlchemy extension

The file `app.py` :

- Creates a Flask application object named `app`
- Configures the database connection to a local file `app.db`
- Creates an object named `migrate` to manage schema migrations (versions)

- Initializes the SQLAlchemy extension with the application

```
# server/app.py

from flask import Flask
from flask_migrate import Migrate

from models import db

# create a Flask application object
app = Flask(__name__)

# configure a database connection to the local file app.db
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'

# disable modification tracking to use less memory
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

# create a Migrate object to manage schema modifications
migrate = Migrate(app, db)

# initialize the Flask application to use the database
db.init_app(app)


if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Eventually we will add routes to `app.py` to implement CRUD operations on the `pets` table. First, we need to step through the process of creating the database file `app.db` and adding the `pets` table to the database.

Let's configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables before proceeding with the database migration:

```
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

Schema migration with Flask-Migrate

We know how to write SQL statements to define and modify a database schema. For example, we used the SQL `create table` statement to define a database table, and the `update table` statement to modify the structure of an existing table.

A migration is a set of SQL statements that tells a database how to move from an old schema to a new one (schema migration), and also how to move a database entirely from one server to the next (server migration). We will only be focusing on the former, **schema migrations**.

A schema migration is also performed when we first define a schema, i.e. to create the initial tables and other database structures. Thankfully, we can use the Flask extension **Flask-Migrate** to automatically define the schema based on models contained in `server/models.py` !

Change into the `server` directory:

```
$ cd server
```

Type the following command to create a migration environment:

```
$ flask db init
```

The `server` directory should now contain two new directories `instance` and `migrations` :

```
..
├── CONTRIBUTING.md
├── LICENSE.md
├── Pipfile
├── Pipfile.lock
├── README.md
├── notes
└── server
    ├── app.py
```

```
├── instance
├── migrations
|   ├── README
|   ├── alembic.ini
|   ├── env.py
|   ├── script.py.mako
|   └── versions
└── models.py
```

The `instance` folder will eventually hold the database file `app.db` .

The `migrations` folder contains a migration environment:

- `alembic.ini` defines environment configuration options.
- `env.py` defines and instantiates a SQLAlchemy engine, connects to that engine, starts a transaction, and calls the migration engine.
- `script.py.mako` is a template that is used when creating a migration- it defines the basic structure of a migration.
- `versions` is a directory to hold migration scripts.

Next we will use the command `flask db migrate -m message` to generate a migration script in the `migrations/versions` directory. The `-m message` is an optional flag that lets us add a message describing the migration.

Type the following command to generate an initial migration:

```
$ flask db migrate -m "Initial migration."
```

. The command results in two new files:

- The database `app.db` is added to the `instance` directory.
- A Python migration script of the form `###_message.py` is added to the `migrations/versions` directory.
  - `###` is a random version number.
  - `message` is the text specified with the `-m` flag.

```
├── CONTRIBUTING.md
├── LICENSE.md
```

```
├── Pipfile
├── Pipfile.lock
├── README.md
├── notes
└── server
    ├── app.py
    ├── instance
    |   └── app.db
    ├── migrations
    |   ├── README
    |   ├── alembic.ini
    |   ├── env.py
    |   ├── script.py.mako
    |   └── versions
    |       └── ###_message.py
    └── models.py
```

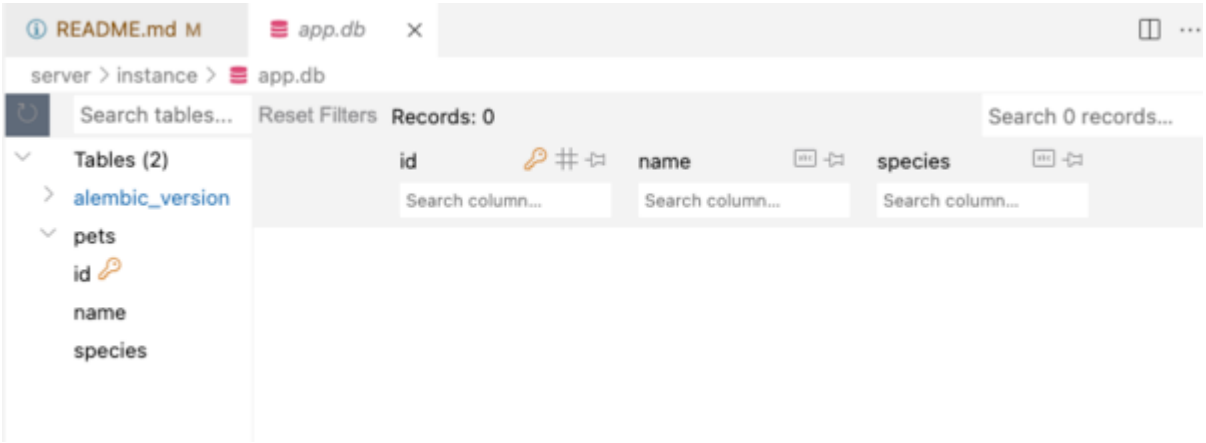
Open the migration script in the editor. You'll see it contains functions `upgrade()` and `downgrade()` that create and drop the `pets` table. The `upgrade()` function is generated using the schema details defined by the `Pet` model class.

Finally, type the following to run the `upgrade()` function and create the `pets` table:

```
$ flask db upgrade head
```

The `head` is optional and refers to the most recent migration version.

Open the database file `app.db` using a VSCode extension for viewing SQLite database. The image below shows the database using the `SQLite Viewer` extension. Assuming you've installed the extension, right-click on `app.db`, then select `Open With.../SQLite Viewer`. Confirm the database contains a new table named `pets` with columns as defined by the `Pet` model class.



Column Constraints

The `Pet` model maps to a basic database table with 3 columns (`id` , `name` , `species`). There are no column constraints other than the primary key `id` :

```
# define a model class by inheriting from db.Model.
class Pet(db.Model):
    __tablename__ = 'pets'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String)
    species = db.Column(db.String)
```

However, SQLAlchemy (and therefore Flask-SQLAlchemy) let's us define many types of column constraints. For example, the `User` model below demonstrates some common constraints:

```
class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True,
```

```
                        nullable=False, index=True)
    email = db.Column(db.String(120), unique=True)
    verified = db.Column(db.Boolean, default=False)
```

- `id` is the primary key
- `username` is a unique string of length 80. Null values are not allowed. An index is set on the column to speed up queries when searched by this column.
- `email` is a unique string of length 120. Null values are allowed.
- `verified` is a boolean that defaults to `False` if a value is not given.

Conclusion

Flask-SQLAlchemy is a Flask extension that simplifies the task of connecting a Flask application with a database. Flask-SQLAlchemy also makes it easy to define a database model.

Flask-Migrate is a Flask extension for performing schema migrations. We used several Flask-Migrate commands to create the initial version of our database:

Flask-Migrate Command	What It Does	Command Frequency
`flask db init`	Creates a new migration environment in `migrations` folder	Run once
`flask db migrate`	Autogenerates a new migration script in the `migrations` folder.	Run each time a model class is added/updated in `models.py` .
`flask db upgrade head`	Runs the `upgrade` function to update the database schema.	Run after creating a new migration script.

Resources

- **SQLAlchemy (https://www.sqlalchemy.org/)**

- **Alembic (https://alembic.sqlalchemy.org/en/latest/)**
- **Flask-SQLAlchemy (https://flask-sqlalchemy.palletsprojects.com/en/3.0.x/)**
- **Flask-Migrate (https://flask-migrate.readthedocs.io/en/latest/)**
- **SQLite Viewer VSCode Extension (https://marketplace.visualstudio.com/items?itemName=qwtel.sqlite-viewer)**
- **Metadata - SQLAlchemy 2.0 Documentation (https://docs.sqlalchemy.org/en/20/core/metadata.html)**

This tool needs to be loaded in a new browser window

Load Flask-SQLAlchemy : Code-Along (CodeGrade) in a new window